



Hinweise für die Klausur

Prüfung (1/3)

- **4.2.2026 um 10 Uhr in E46 und E62**
- Raumaufteilung schicke ich am 2.2.2026!
- Wie läuft es ab?
 - 10 Uhr: Ausgabe der Prüfungsaccounts
 - Login im eAssessment
 - Etwa 10:15 Uhr: Start der Prüfung
 - Nach der vorgegebenen Zeit (120 Min) wird die Klausur im eAssessment automatisch abgegeben (Ergebnisse immer schnell dort eintragen)
 - Ende der Prüfung etwa 12:15 Uhr

Prüfung (2/3)

- **Meldet euch jetzt an!**
- Anmeldung bis **1.2.2026 bis 23:59:59** in myHAW
 - Danach werden eure Prüfungsaccounts erstellt
- Abmeldung bis **1.2.2026 bis 23:59:59** in myHAW

Prüfung (3/3)

- 120 Minuten (plus Nachteilsausgleich obvsl.)
- Hilfsmittel:
 - DinA4-Zettel, beidseitig beschrieben (von Hand oder gedruckt)
 - Regex cheat sheet und Tastaturbelegungssheet
 - **Sonst nichts!**
- Aufgabenart: Theorie Multiple Choice und Coding (90 % Coding)

Schaut euch genauer an:

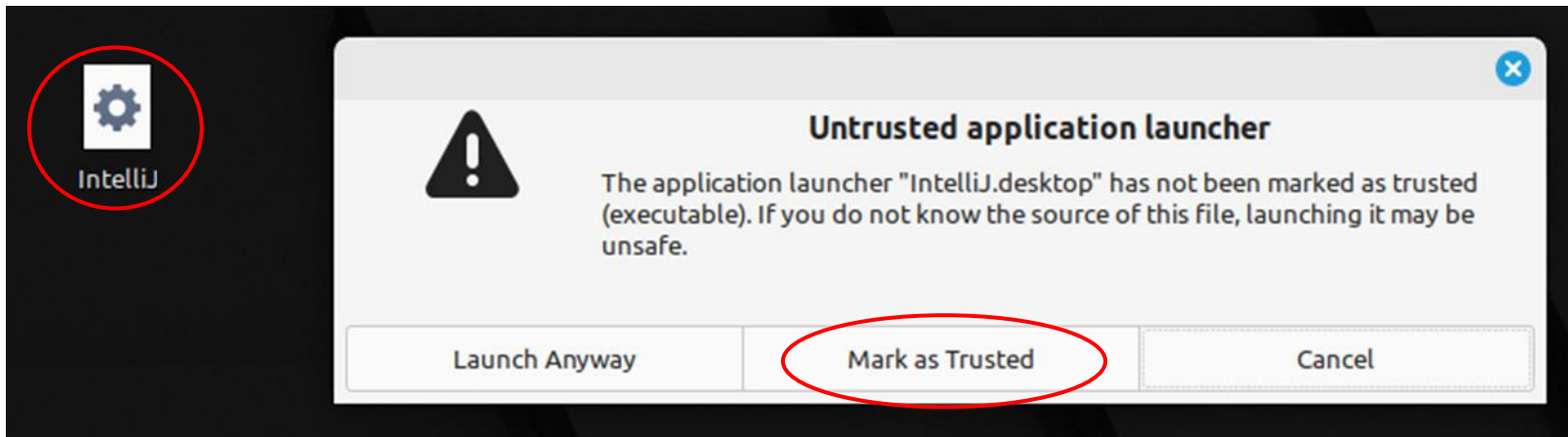
- Callbacks (Übungen Callbacks)
- Collections (Übungen Collections)
- Concurrency (Übungen Concurrent)
- Generics (Übungen Generic Method und Types)
- Parser (Übungen Regex)
- Rekursion (Übungen Rekursion)
- Lambda und Streams (Übungen Lambda und Streams)
- Unit Tests (Übungen Unit Test)
- Exceptions (Übungen Exceptions und Optional)

Dies sind Schwerpunkte!
Nicht alle diese Themen kommen dran!
Es sind nicht NUR diese Themen.

Wenn ihr diese Themen beherrscht ist
die Klausur auf jeden Fall bestanden ;)

IntelliJ starten

Auf dem Desktop befindet sich eine Verknüpfung "IntelliJ.desktop".
Sobald man einen Doppelklick darauf macht, öffnet sich folgender Dialog:



Im Dialog auf "Mark as Trusted" klicken.

Anschließend verschwindet der Dialog und das Symbol von IntelliJ auf dem Desktop verändert sich zu:  .
Jetzt kann IntelliJ mit einem Doppelklick geöffnet werden.

Klausur-Projekt herunterladen

Lade die für die Klausur zur Verfügung gestellte zip-Datei im [e-Assessment](#) herunter.
Hinweis: Der Download ist nur während der Klausur verfügbar.

Verbleibende Zeit 1:59:33

Information

Frage markieren

Frage bearbeiten

Quelltext-Download

[Klicke hier, um das Klausur-Projekt herunterzuladen](#) <--- click me!

Vorherige Seite

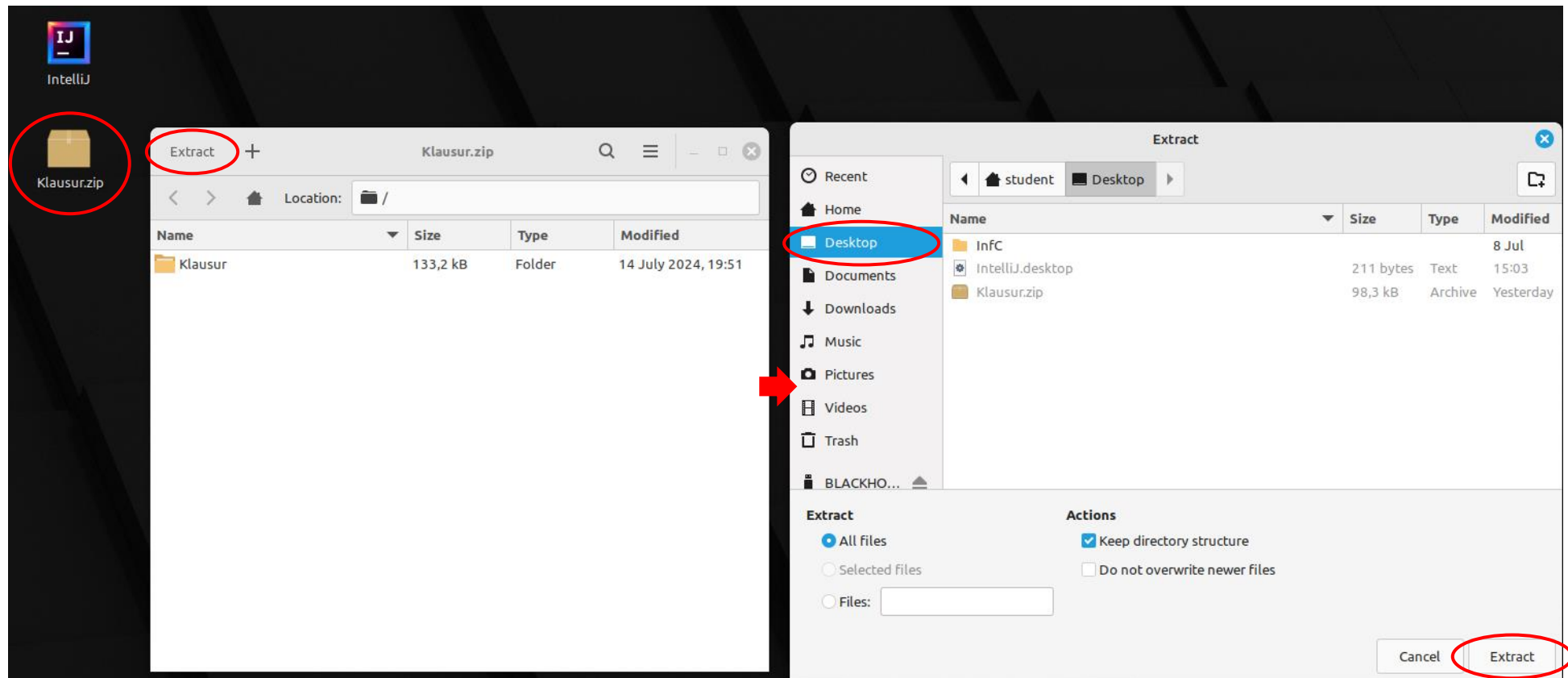
Nächste Seite

Klausur-Projekt entpacken

Doppelklicke auf die heruntergeladene zip-Datei ("Klausur.zip").

Es öffnet sich die App "Archive Manager". Klicke "Extract".

Entpacke den Inhalt der zip-Datei irgendwo auf dem Rechner, z. B. Home oder Desktop.



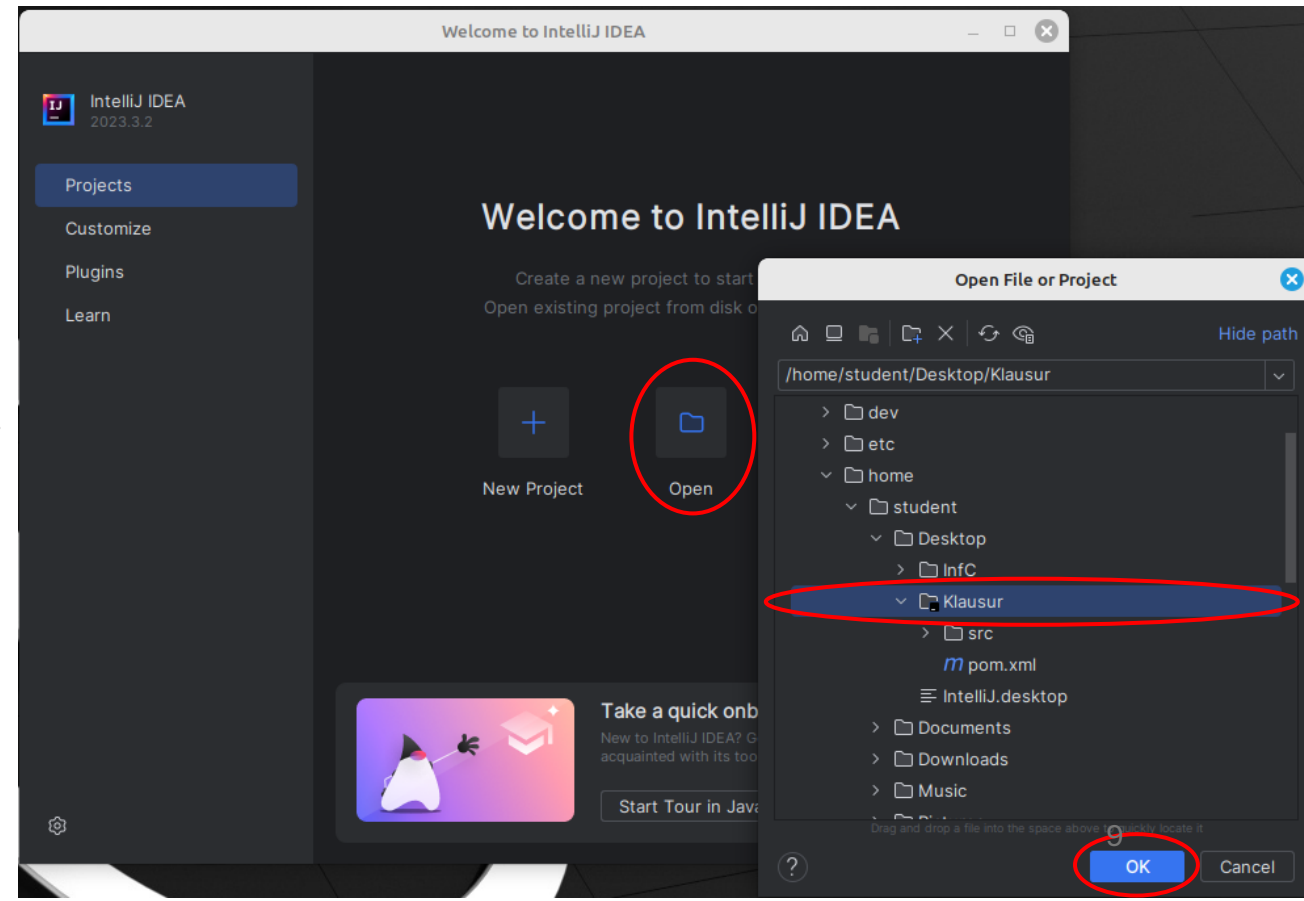
IntelliJ-Projekt öffnen

Klicke Open in IntelliJ und navigiere zum entpackten Ordner. Klicke darauf, um den Ordner zu markieren. Klicke OK, um das Klausur-Projekt zu öffnen.

Bestätige die folgenden Dialoge:

- Terms → OK.
- Privacy → Don't send.
- Trust → Yes.

Nachdem IntelliJ das Projekt geöffnet hat, muss du eventuell das SDK (siehe Banner oben rechts) auf Temurin 21 setzen (einzige vorhandene Option).



JDK not defined

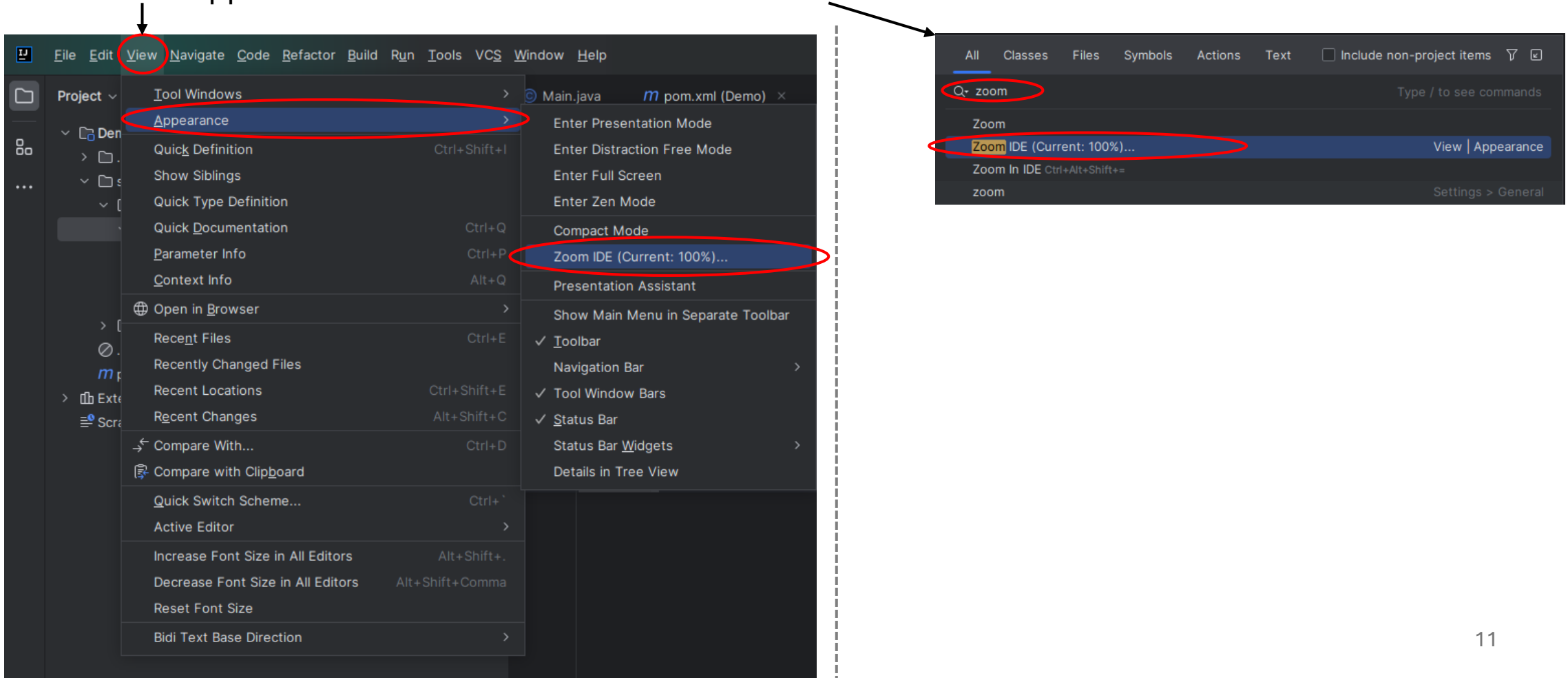
Falls das SDK nicht definiert ist, klicke auf "Setup SDK" und wähle das vorhandene SDK mit der höchsten Version. Anschließend dauert es einen kleinen Moment, bis alle Dateien indiziert sind.



Zoom

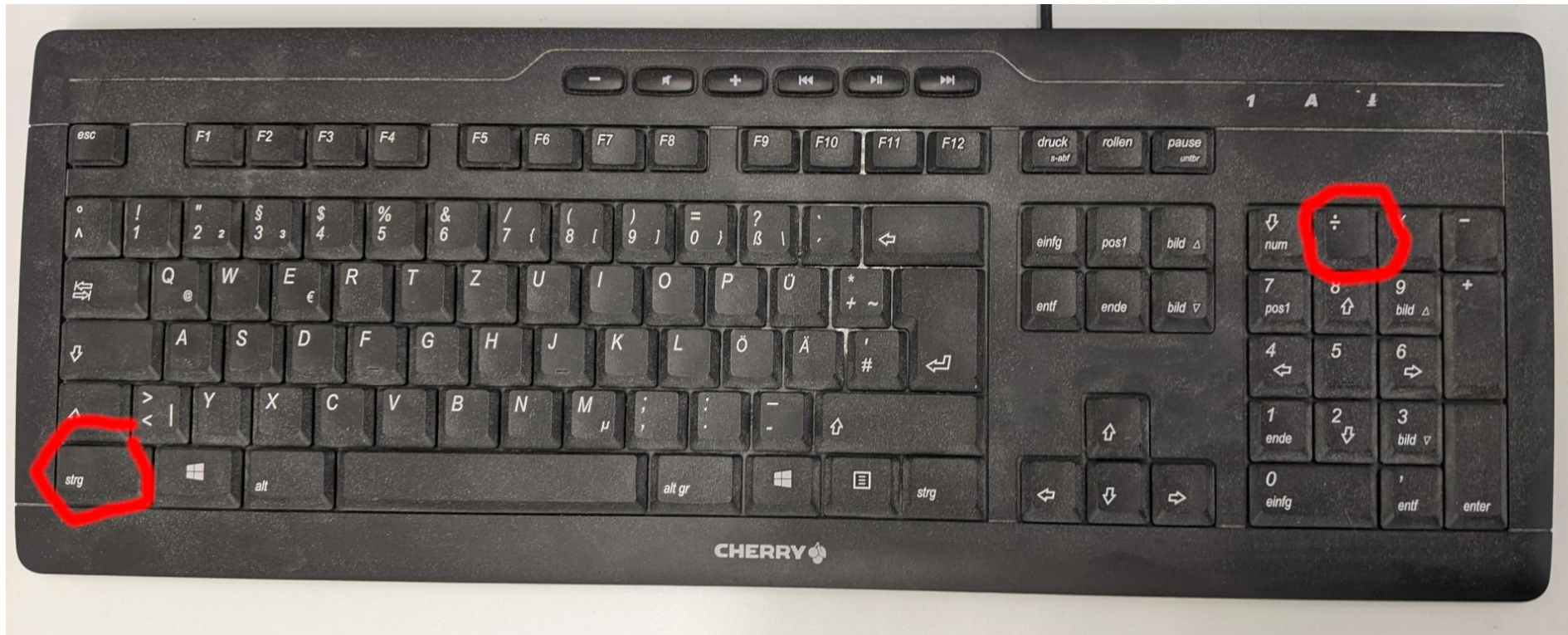
Bei manchen Systemen, ist die Schrift klein.

Über "View->Appearance->Zoom IDE" oder "SHIFT"+"SHIFT"+"Zoom IDE" kann die Zoomstufe erhöht werden.

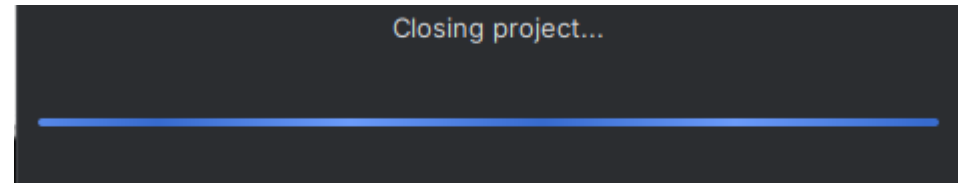


Tastaturkürzel: Ein- und Auskommentieren

Standardmäßig ist bei IntelliJ das Tastaturkürzel zum Ein-/Auskommentieren STRG + /
Auf den Prüfungsrechnern funktioniert das Kürzel nur über den Zahlenblock (NumPad) (siehe Foto).

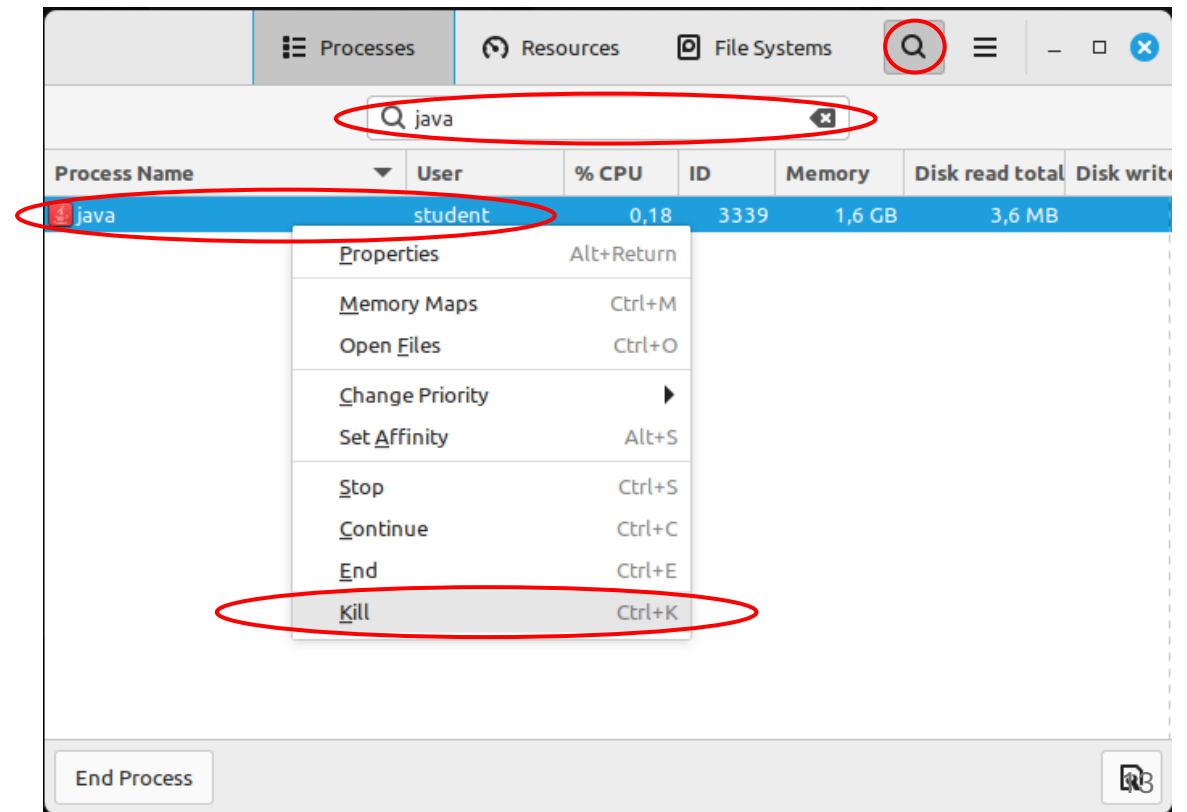
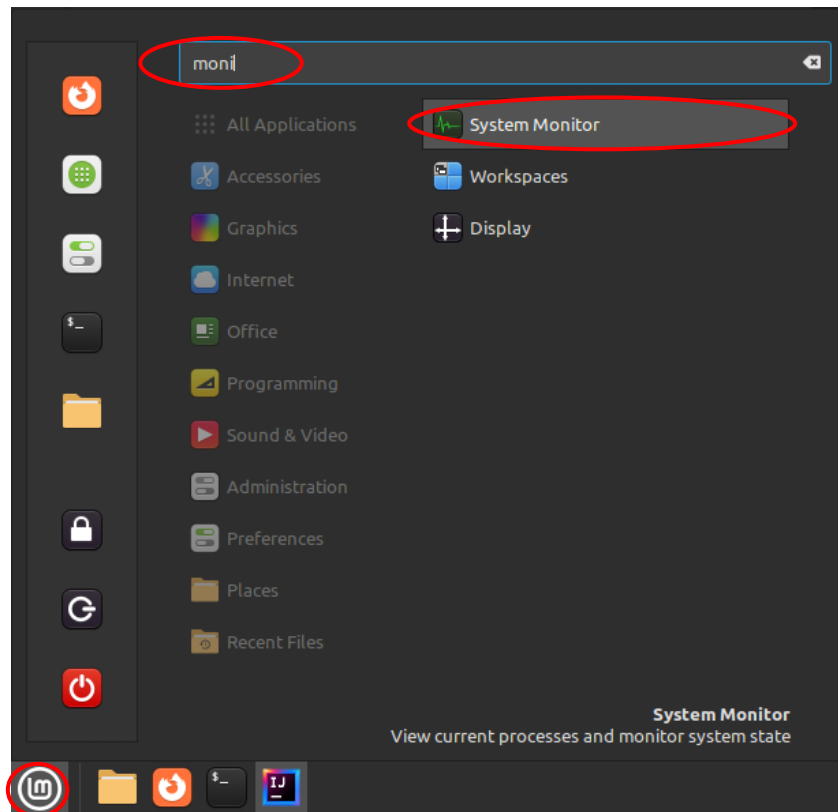


IntelliJ beenden

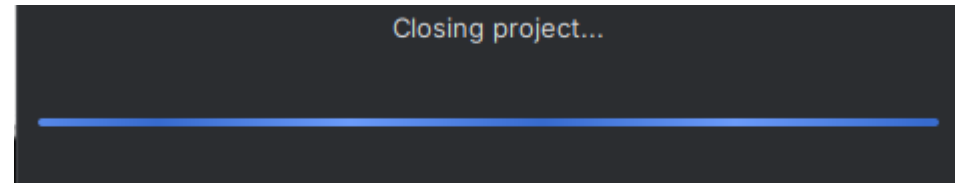


Vermeide es, IntelliJ zu schließen oder ein geöffnetes IntelliJ-Projekt zu schließen. Eventuell folgt ein "Closing project..."-Dialog, der nicht mehr verschwindet.

In dem Fall muss IntelliJ mit dem Programm "System Monitor" beendet werden:



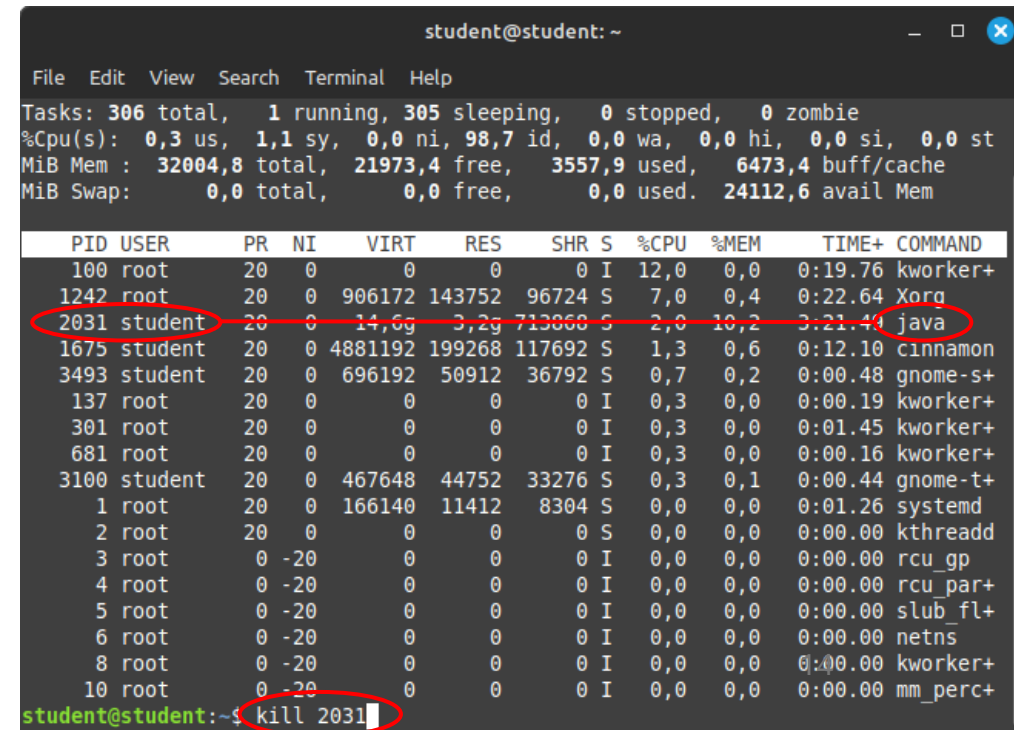
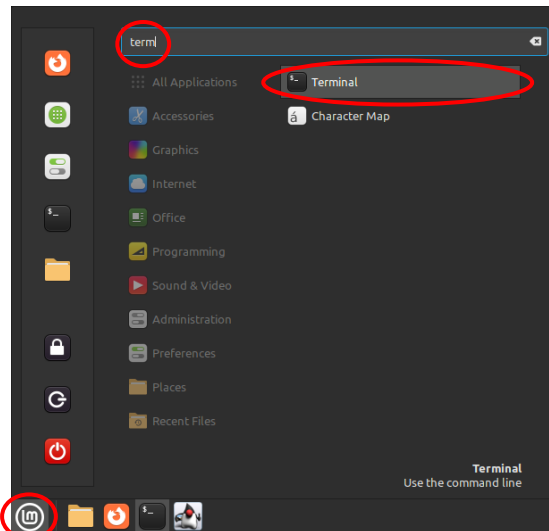
IntelliJ beenden



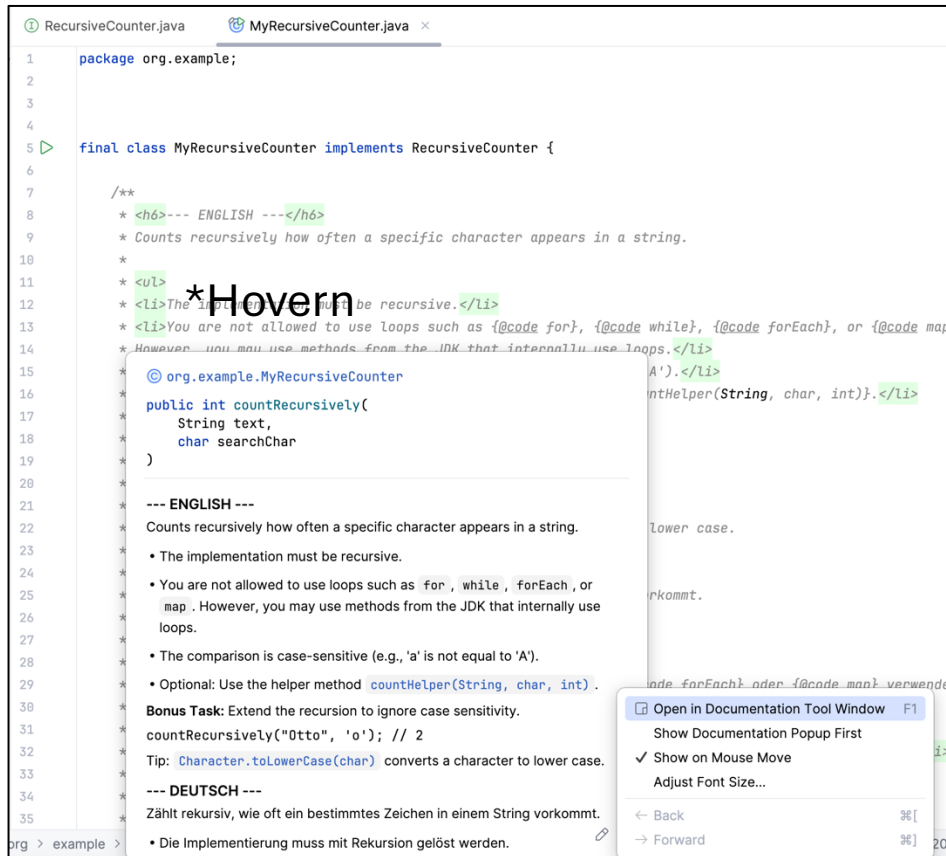
Vermeide es, IntelliJ zu schließen oder ein geöffnetes IntelliJ-Projekt zu schließen. Eventuell folgt ein "Closing project..."-Dialog, der nicht mehr verschwindet.

In dem Fall muss IntelliJ mit kill in der Kommandozeile beendet werden:

- Start->Terminal (öffne ein Terminal)
- > top (finde die PID des "java"-Prozesses, i.d.R. oben rechts)
- q (um top zu verlassen)
- > kill 2031 (ersetze die Zahl mit der PID)



Aufgabe neben dem Code anzeigen

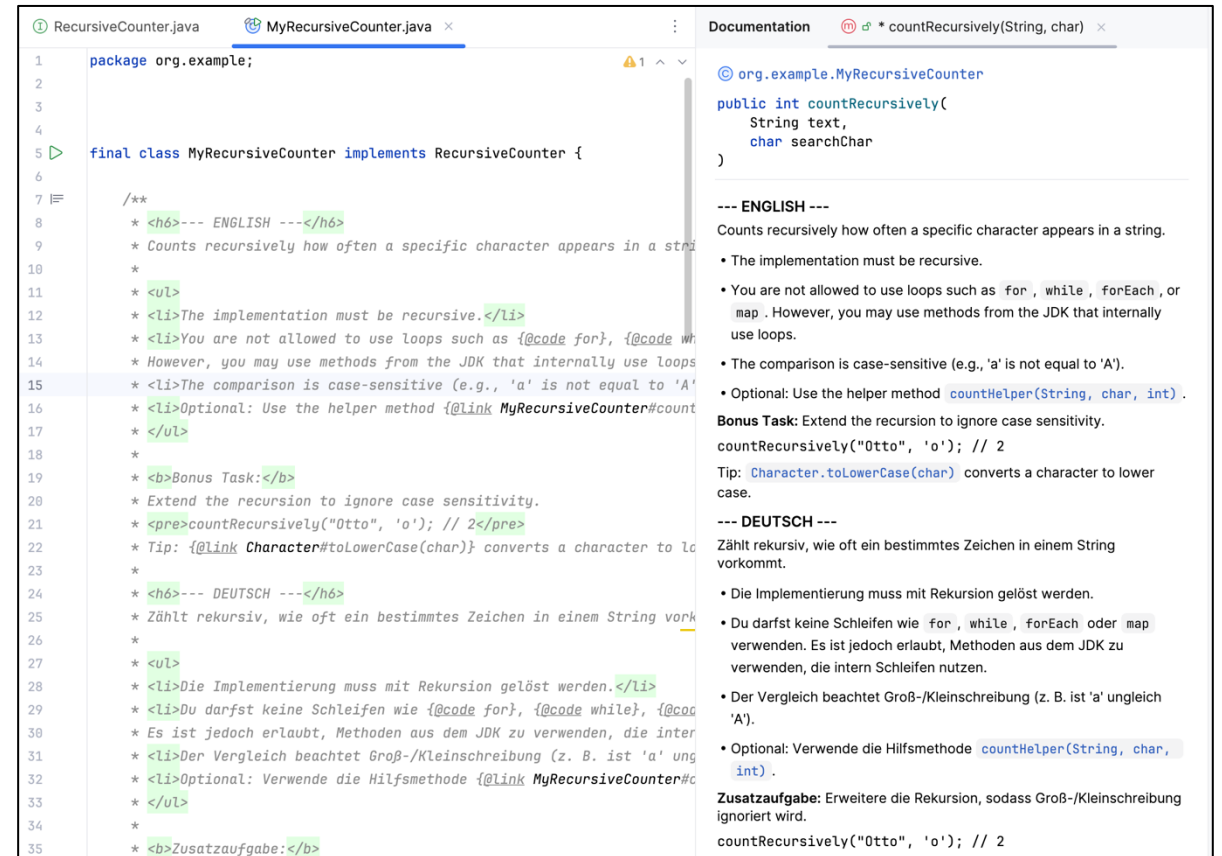


The screenshot shows an IDE with a Java file named `MyRecursiveCounter.java`. The code defines a class `MyRecursiveCounter` that implements `RecursiveCounter`. A Javadoc comment is visible, describing the `countRecursively` method. A hover tooltip is displayed over the Javadoc comment, showing the full text of the comment in English and German. The tooltip also includes a 'Bonus Task' and a 'Tip'.

```
package org.example;

final class MyRecursiveCounter implements RecursiveCounter {

    /**
     * <h6>--- ENGLISH ---</h6>
     * Counts recursively how often a specific character appears in a string.
     *
     * <ul>
     * <li>The implementation must be recursive.</li>
     * <li>You are not allowed to use loops such as {code for}, {code while}, {code forEach}, or {code map}. However, you may use methods from the JDK that internally use loops.</li>
     * </ul>
     *
     * <li>The comparison is case-sensitive (e.g., 'a' is not equal to 'A').
     * <li>Optional: Use the helper method {code countHelper(String, char, int)}.</li>
     * </ul>
     *
     * --- ENGLISH ---
     * Counts recursively how often a specific character appears in a string.
     *
     * • The implementation must be recursive.
     *
     * • You are not allowed to use loops such as for , while , forEach , or map . However, you may use methods from the JDK that internally use loops.
     *
     * • The comparison is case-sensitive (e.g., 'a' is not equal to 'A').
     *
     * • Optional: Use the helper method countHelper(String, char, int) .
     *
     * Bonus Task: Extend the recursion to ignore case sensitivity.
     * countRecursively("Otto", 'o'); // 2
     * Tip: Character.toLowerCase(char) converts a character to lower case.
     *
     * --- DEUTSCH ---
     * Zählt rekursiv, wie oft ein bestimmtes Zeichen in einem String vorkommt.
     *
     * • Die Implementierung muss mit Rekursion gelöst werden.
     *
     * • Du darfst keine Schleifen wie for , while , forEach oder map verwenden. Es ist jedoch erlaubt, Methoden aus dem JDK zu verwenden, die intern Schleifen nutzen.
     *
     * • Der Vergleich beachtet Groß-/Kleinschreibung (z. B. ist 'a' ungleich 'A').
     *
     * • Optional: Verwende die Hilfsmethode countHelper(String, char, int) .
     *
     * Zusatzaufgabe: Erweitere die Rekursion, sodass Groß-/Kleinschreibung ignoriert wird.
     * countRecursively("Otto", 'o'); // 2
     *
     * <b>Zusatzaufgabe:</b>
     */
    public int countRecursively(
        String text,
        char searchChar
    ) {
        // Implementation
    }
}
```



The screenshot shows the same IDE with the `MyRecursiveCounter.java` file. The documentation window is open, displaying the Javadoc comment for the `countRecursively` method. The documentation is presented in a structured format, including English and German descriptions, a list of requirements, a bonus task, and a tip.

```
package org.example;

final class MyRecursiveCounter implements RecursiveCounter {

    /**
     * <h6>--- ENGLISH ---</h6>
     * Counts recursively how often a specific character appears in a string.
     *
     * <ul>
     * <li>The implementation must be recursive.</li>
     * <li>You are not allowed to use loops such as {code for}, {code while}, {code forEach}, or {code map}. However, you may use methods from the JDK that internally use loops.</li>
     * </ul>
     *
     * <li>The comparison is case-sensitive (e.g., 'a' is not equal to 'A').
     * <li>Optional: Use the helper method {code countHelper(String, char, int)}.</li>
     * </ul>
     *
     * --- ENGLISH ---
     * Counts recursively how often a specific character appears in a string.
     *
     * • The implementation must be recursive.
     *
     * • You are not allowed to use loops such as for , while , forEach , or map . However, you may use methods from the JDK that internally use loops.
     *
     * • The comparison is case-sensitive (e.g., 'a' is not equal to 'A').
     *
     * • Optional: Use the helper method countHelper(String, char, int) .
     *
     * Bonus Task: Extend the recursion to ignore case sensitivity.
     * countRecursively("Otto", 'o'); // 2
     * Tip: Character.toLowerCase(char) converts a character to lower case.
     *
     * --- DEUTSCH ---
     * Zählt rekursiv, wie oft ein bestimmtes Zeichen in einem String vorkommt.
     *
     * • Die Implementierung muss mit Rekursion gelöst werden.
     *
     * • Du darfst keine Schleifen wie for , while , forEach oder map verwenden. Es ist jedoch erlaubt, Methoden aus dem JDK zu verwenden, die intern Schleifen nutzen.
     *
     * • Der Vergleich beachtet Groß-/Kleinschreibung (z. B. ist 'a' ungleich 'A').
     *
     * • Optional: Verwende die Hilfsmethode countHelper(String, char, int) .
     *
     * Zusatzaufgabe: Erweitere die Rekursion, sodass Groß-/Kleinschreibung ignoriert wird.
     * countRecursively("Otto", 'o'); // 2
     *
     * <b>Zusatzaufgabe:</b>
     */
    public int countRecursively(
        String text,
        char searchChar
    ) {
        // Implementation
    }
}
```

Die Aufgaben sind in Javadoc im Code geschrieben und können dementsprechend angezeigt werden!